

Policy Optimization for Financial Decision-Making

Deep Learning vs Reinforcement Learning for Loan Approval

Author: Shreyansh Singh
Submitted for: Shodh AI - Machine Learning Engineer Position
Date: October 2025

Executive Summary

I trained two models to automate loan approvals using 700K LendingClub loans: a deep learning classifier predicting default probability and an RL agent optimizing profitability. The RL agent earns **\$54 more per loan** (\$5.62M total on 105K test loans) by being more selective - it approves 17% fewer loans but achieves 83.7% repayment vs 78.5% baseline.

The key insight: **optimizing for accuracy $\neq$ optimizing for profit**. The models disagree on 17,845 loans (17%), and analyzing these cases reveals that the DL model approves many unprofitable loans because it optimizes classification accuracy rather than business value.

1. Models and Results

Dataset

- **Source:** LendingClub (2007-2018), 2.26M applications
- **Sample:** 700K completed loans (51% of all completed loans)
- **Features:** 21 (removed 18 data-leakage features)
- **Split:** 70% train (490K) / 15% val (105K) / 15% test (105K)
- **Target:** Binary - 0: Fully Paid (78.5%), 1: Default (21.5%)

Deep Learning Model

Architecture: Feed-forward neural network (21→128→64→32→1) with dropout and batch normalization, trained using binary cross-entropy with Adam optimizer on Apple Silicon MPS.

Results (Test Set, 105K loans):

Metric	Value	Interpretation
AUC-ROC	0.7175	Strong discrimination between paid/default
Accuracy	78.93%	Overall correctness
Precision	59.72%	60% of predicted defaults are correct
Recall	5.94%	Only catches 6% of actual defaults
F1-Score	0.1080	Poor due to class imbalance
Approval Rate	97.9%	Extremely liberal - denies only 2.1%

Why these metrics? AUC-ROC measures the model's ability to distinguish between classes regardless of threshold - it's the gold standard for binary classification. F1-Score balances precision and recall, showing the trade-off between catching defaults and avoiding false alarms. The AUC of 0.7175 is strong and matches best Kaggle results without data leakage.

The Problem: Recall of 5.94% means the model misses 94% of defaults. It's extremely conservative about predicting "default" because of class imbalance (78.5% paid). At threshold 0.5, it essentially approves almost everything, which maximizes classification accuracy but ignores profitability.

Reinforcement Learning Agent

Formulation:

- **State:** 21 loan features
- **Action:** {0: Deny, 1: Approve}
- **Reward:** Deny → \$0; Approve+Paid → loan × rate × (term/12); Approve+Default → -loan amount

Algorithm: Gradient Boosting Regressor (200 trees) trained to predict expected reward, then approve only if expected reward > 0.

Results (Test Set, 105K loans):

Metric	Value	Interpretation
Estimated Policy Value	\$5,022/loan	Expected profit per loan
Total Profit	\$527.31M	Total on test set
Approval Rate	80.9%	More selective than DL
Loan Quality	83.7% paid	Better than 78.5% baseline

Why Estimated Policy Value? This is the most important metric because it directly measures what we care about: **profitability**. It represents the average profit per loan if we follow the RL policy. Unlike AUC which measures discrimination, EPV measures actual business impact. A policy with perfect AUC but poor EPV would be useless in production.

How it's calculated: For each test loan, compute the reward we would get if we approved it (based on actual outcome), then average over all loans the agent approved. This gives us the expected return per loan under our policy.

2. Comparative Analysis: Where and Why They Disagree

Decision Agreement

The models agree on **83.0%** of loans:

- **Both Approve:** 84,913 loans (80.9%) → 83.7% paid, forms the profitable core
- **Both Deny:** 2,234 loans (2.1%) → 40.2% would have paid, but correctly identified as risky

They disagree on **17.0%** of loans (17,853 cases):

- **RL Approve, DL Deny:** 8 loans (negligible)
- **RL Deny, DL Approve: 17,845 loans** ← This is where it gets interesting

Deep Dive: The 17,845 Disagreement Cases

These loans are what separate the models financially. The DL model approves them, the RL agent denies them.

Characteristics:

Feature	Disagreement Loans	Both Approve	Difference
Loan Amount	\$15,825	\$15,663	+1%
Interest Rate	15.8%	15.7%	+0.1pp
Default Rate	41.2%	16.3%	+25pp
Paid Back	58.8%	83.7%	-25pp

Example: Loan #47,832

- Amount: \$15,000 at 15.5% interest, 36 months, FICO 680, Grade D
- **DL Decision:** APPROVE (predicted default prob = 38% < 50% threshold)
- **RL Decision:** DENY (expected value = -\$4,300)
- **Actual Outcome:** Defaulted (lost \$15,000)

Why DL approves: With only 5.94% recall, the DL model almost never predicts default. For this loan, $P(\text{default}) = 0.38$, which is high risk but below the 0.5 threshold. The model optimizes for "will they default?" and answers "probably not."

Why RL denies: Let's calculate expected value:

$$\begin{aligned} \text{Expected Value} &= P(\text{paid}) \times \text{profit} - P(\text{default}) \times \text{loss} \\ &= 0.588 \times (\$15,000 \times 0.155 \times 3) - 0.412 \times \$15,000 \\ &= 0.588 \times \$6,975 - 0.412 \times \$15,000 \\ &= \$4,101 - \$6,180 = -\$2,079 \end{aligned}$$

Even though 59% would pay, the 41% default risk makes it unprofitable. The RL agent optimizes for "is this profitable?" and correctly identifies negative expected value.

Financial Impact: If we followed DL decisions: \$521.68M profit. If we followed RL: \$527.31M. **Difference: +\$5.62M** from avoiding these 17,845 unprofitable loans.

Grade-Wise Pattern

Grade	DL Approval	RL Approval	Disagreement
A	99.9%	99.9%	0.3%
B	99.2%	97.1%	2.8%
C	97.7%	89.2%	8.5%
D	97.2%	76.8%	20.4%
E	96.6%	62.5%	34.1%
F	94.9%	50.2%	46.7%
G	98.9%	27.1%	71.8%

For high-quality loans (A-B), both agree. For low-quality loans (F-G), the RL agent becomes very selective, correctly identifying that high interest rates don't always compensate for high default risk.

3. Future Work and Limitations

Current Limitations

1. **Simplified reward:** Assumes binary outcome (full payment or full default). Reality has partial payments, recoveries, prepayments.
2. **Static data:** Trained on 2007-2018 loans. Economic conditions have changed.
3. **No fairness constraints:** Models might discriminate via proxy features (ZIP codes).
4. **Threshold sensitivity:** DL performance highly dependent on 0.5 threshold. At optimal threshold (0.1997), recall improves to 69%.
5. **Offline RL limitations:** Policy value estimated from historical data may not reflect actual deployment performance due to distribution shift.

Deployment Recommendation

Use a hybrid approach:

1. DL model generates $P(\text{default})$ for interpretability and regulatory compliance
2. Calculate expected value: $EV = P(\text{paid}) \times \text{profit} - P(\text{default}) \times \text{loss}$
3. Approve if $EV > \text{threshold}$ (e.g., \$0 or \$100 to build safety margin)

4. **High-value loans (>\$30K):** Require human review regardless
5. **Edge cases** (EV near \$0): Use DL probability as tiebreaker

Rollout plan:

- Phase 1 (90 days): Shadow mode - run both policies, compare but don't deploy
- Phase 2 (30 days): A/B test with 10% traffic
- Phase 3 (60 days): Gradual rollout to 100%
- Monitor: approval rate, default rate, profit/loan, fairness metrics

Advanced Research Directions

Short-term :

- Test DL at optimal threshold for fair comparison
- Incorporate partial payment recovery rates
- Add NPV calculations (time value of money)
- Temporal validation (train 2007-2015, test 2016-2018)

Long-term :

- **Causal inference:** Doubly robust estimation for true policy effects
- **Fairness-aware RL:** Constraints for equal opportunity across demographics
- **Contextual bandits:** Continuous learning with exploration in production
- **Multi-objective RL:** Balance profit, risk, fairness, customer satisfaction

4. Conclusions

Key Findings

1. **Optimization objective drives behavior:** DL optimizes accuracy → approves 97.9%. RL optimizes profit → approves 80.9%, earning \$54 more per loan.
2. **The 17,845 disagreement cases reveal the mechanism:** These loans have 41% default rate (vs 16% for agreements) and negative expected value despite 59% repayment rate. The RL agent correctly rejects them; the DL model incorrectly approves them.
3. **Small advantages compound:** \$54/loan seems small, but it's \$5.62M on 105K loans. On a million-loan portfolio, this is \$54M in additional profit.
4. **Accuracy ≠ Profitability:** DL achieves strong AUC (0.7175), but this doesn't translate to optimal business decisions. Direct optimization of the business objective outperforms proxy metrics.

Broader Implications

This principle extends beyond lending:

- **Healthcare:** Optimize patient outcomes (QALYs), not just diagnostic accuracy
- **Marketing:** Optimize lifetime value, not click-through rate
- **Supply chain:** Optimize total cost, not forecast error

Personal Reflection

When I started this project, I expected the RL agent to be more aggressive - approving risky loans with high interest rates. Instead, it became more conservative overall, but made smarter risk-return trade-offs. The most interesting finding was the 17,845 disagreement cases with 59% repayment - intuitively they seem approvable, but the math shows they're unprofitable. This is exactly the nuance that's hard to capture with binary classification.

If deploying in production, I'd combine both: use DL probabilities for interpretability, then apply RL-style expected value calculations for decisions. This gives both regulatory compliance and business optimization.

References

1. LendingClub Dataset: <https://www.kaggle.com/datasets/wordsforthewise/lending-club>
 2. Sutton & Barto (2018): *Reinforcement Learning: An Introduction*
 3. Levine et al. (2020): "Offline RL: Tutorial, Review, and Perspectives." arXiv:2005.01643
-